Capture the Flag 6: Active or Inactive?

Goal: Make your Student Directory respond to student data by conditionally showing, hiding, and changing UI elements based on each student's state.

Instructor Note

These activities require your own problem-solving and research skills.
DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

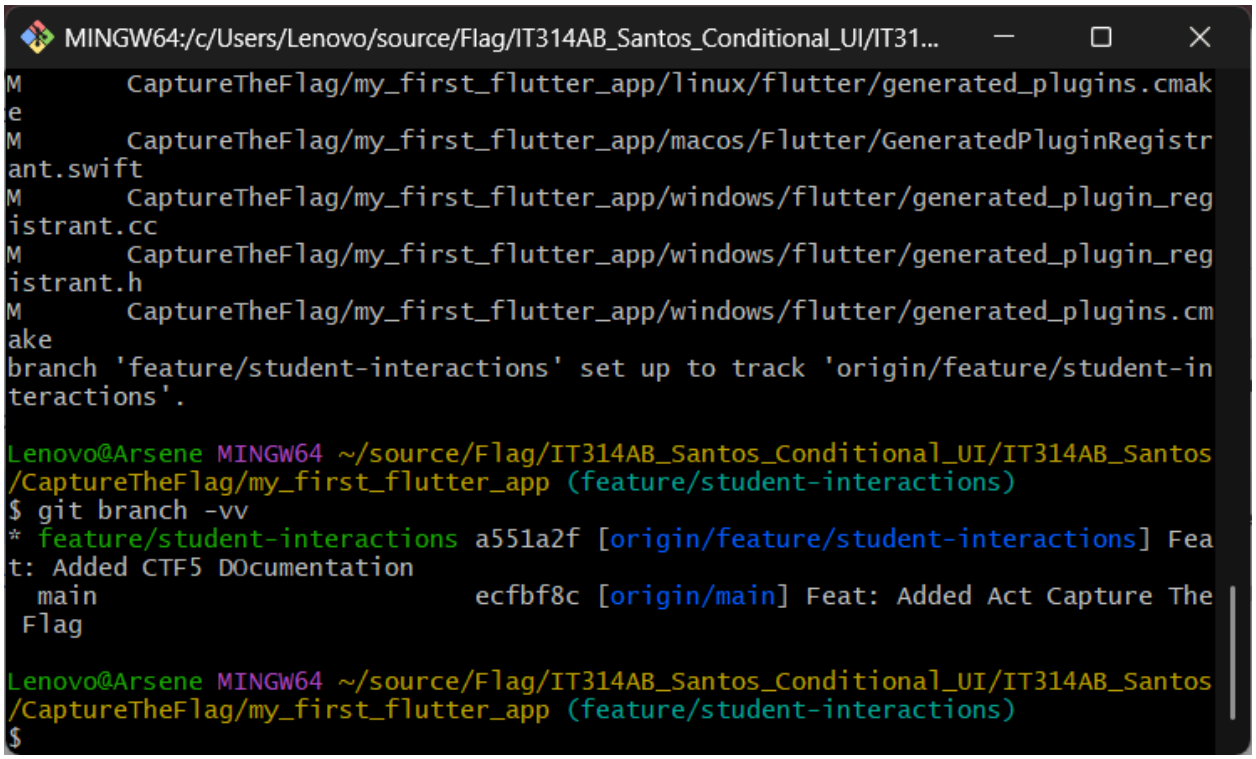
Objective: Create a new feature branch before adding conditional UI.

Create:

`feature/conditional-ui`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Show Active or Inactive

Objective: Change the Student Card based on whether a student is active or inactive.

Add an `active` status to your student data.

Research how Flutter uses **conditional expressions** to display different widgets.

Example states:

- Active
- Inactive

Test

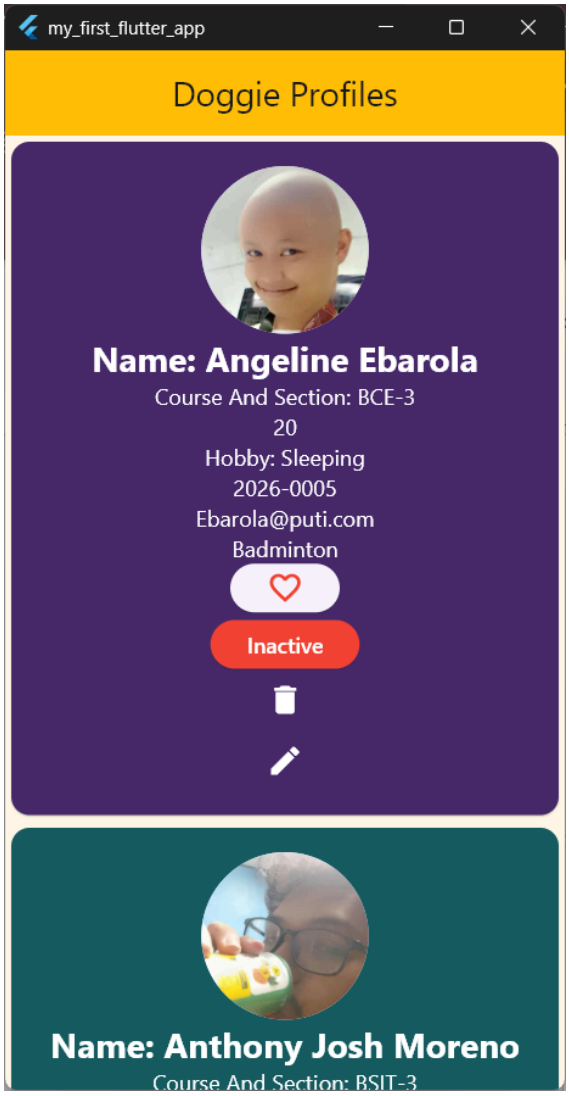
Active students should display an Active indicator.

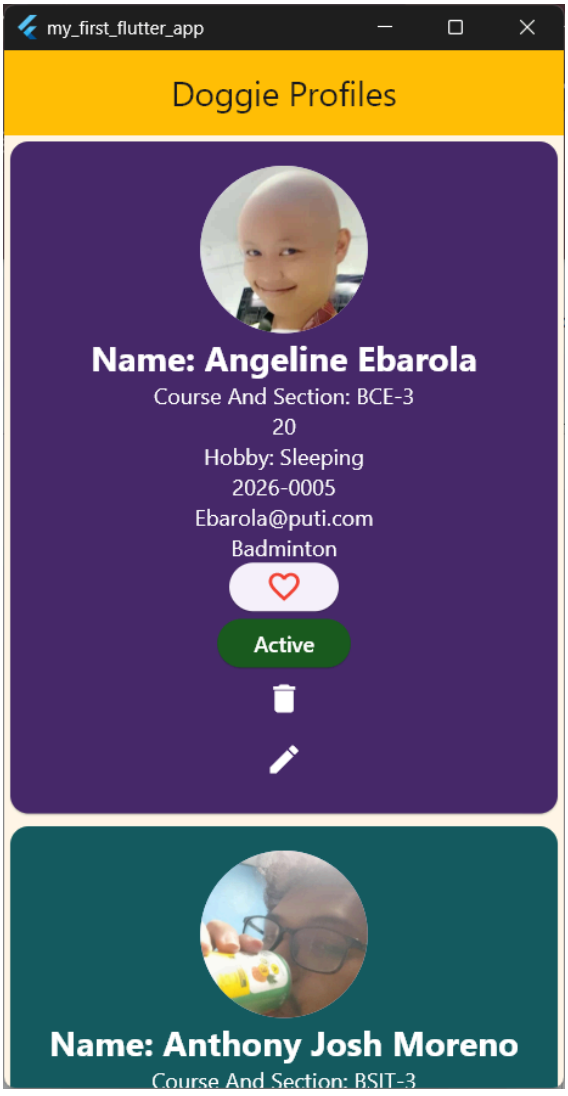
Inactive students should display an Inactive indicator.

Screenshot:

- Active Student Card

- Inactive Student Card
- Conditional logic in your code





```dart
class InteractiveFavButton extends StatefulWidget{
  const InteractiveFavButton({super.key});

  @override
  State<InteractiveFavButton> createState() => _InteractiveFavButtonState();
}

class _InteractiveFavButtonState extends State<InteractiveFavButton>
{
  bool isFavorite = false;
  bool isActive = true;

  @override
  Widget build(BuildContext context) {
    return Column(
      children:[
        ElevatedButton(
          onPressed: () {
            setState(() {
              isFavorite = !isFavorite;
            });
          },
          child: isFavorite ?
          const Icon(
            Icons.favorite,
            color: Colors.red,
            size: 24.0,
            semanticLabel: 'Make it Reddd',
          ) : // Icon
          const Icon(
            Icons.favorite_border,
            color: Colors.red,
            size: 24.0,
            semanticLabel: 'Not red',
          ) // Icon
        ), // ElevatedButton

        const SizedBox(height: 5),
        const SizedBox(height: 5),

        ElevatedButton(
          onPressed: () {
            setState(() {
              isActive = !isActive;
            });
          },
          style: ElevatedButton.styleFrom(
            backgroundColor: isActive ? Colors.green.shade900 : Colors.red,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(50)
            ) // RoundedRectangleBorder
          ),
          child: Text(
            isActive ? "Active" : 'Inactive',
            style: const TextStyle(
              color: Colors.white
            ), // TextStyle
          ), // Text
        ) // ElevatedButton

      ]
    ); // Column
  }
}
```

🏴 Flag 2 — Show or Hide

Objective: Display certain UI elements only when a condition is true.

Add a conditional element to the Student Card.

Examples:

- Favorite badge appears only when favorited
- Warning appears only when inactive
- Delete action appears only if active

Research ways to conditionally include or exclude widgets from the widget tree.

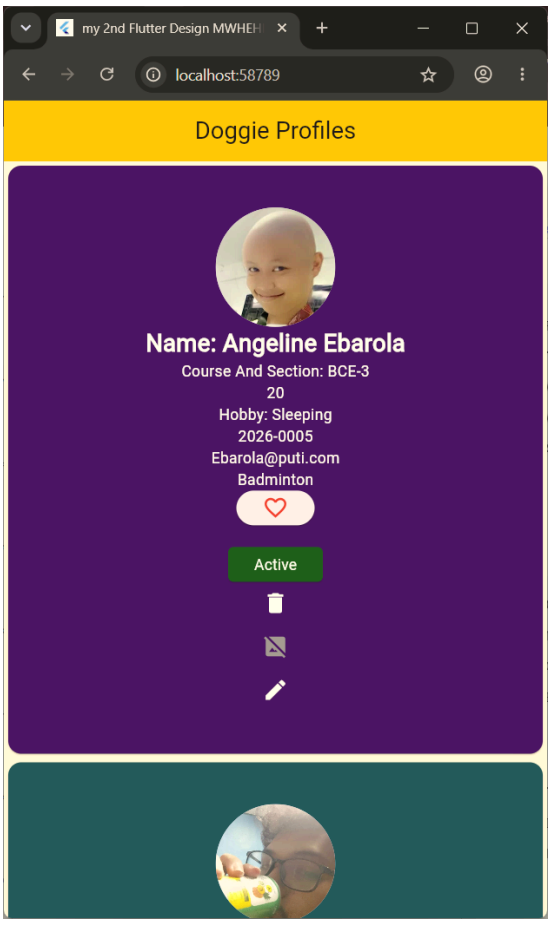
Test

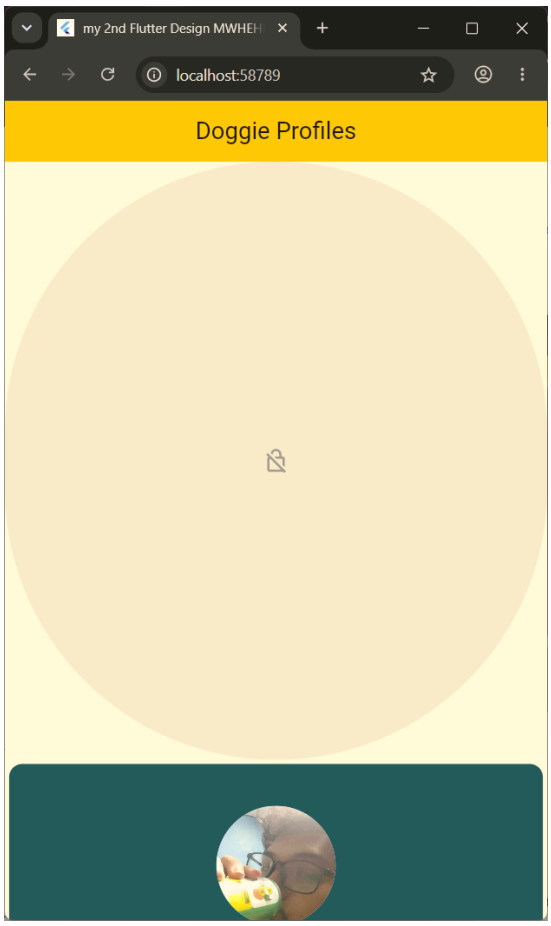
When the condition is true, the element appears.

When the condition is false, the element disappears.

Screenshot:

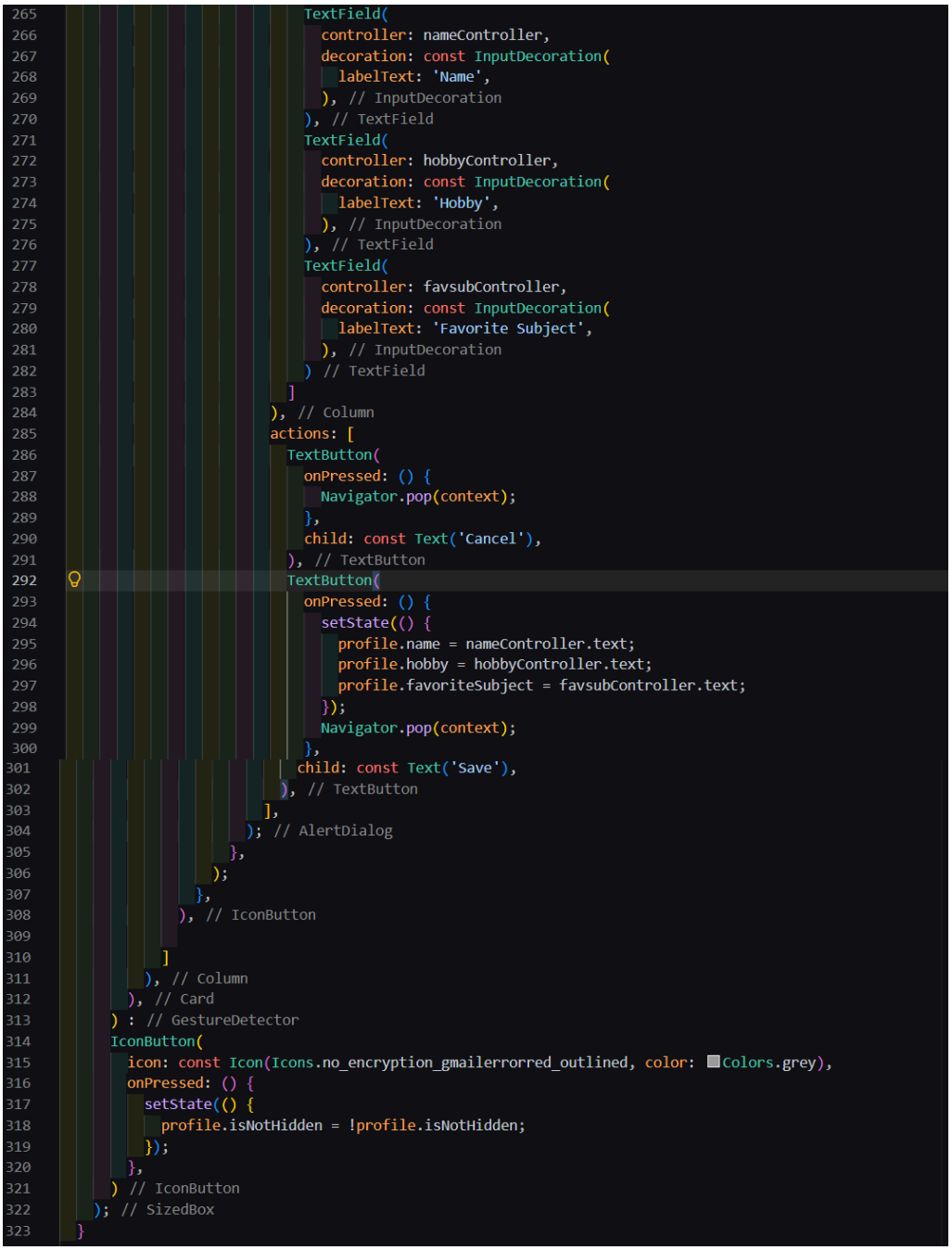
- UI with the element visible
- UI with the element hidden
- Conditional widget code





```dart
class _ProfileState extends State<Profiles>
{
  bool isDeleted = false;

  Widget profilesOfEachUser(Profile profile)
  {
    return SizedBox(
      width: 350,
      height: 550,
      child: profile.isNotHidden ? GestureDetector(
        onTap: ()
        {
          print("Bellooo");    Don't invoke 'print' in production code.⏎Try using a logging framework.
        },
        child: Card(
          color: Color(profile.color),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            crossAxisAlignment: CrossAxisAlignment.center,
            children: [
              CircleAvatar(
                radius: 55,
                backgroundImage: AssetImage(profile.picture),
              ), // CircleAvatar
              Text(profile.name ?? "Name Unknown",
                style: const TextStyle(
                  color : Colors.white,
                  fontSize: 22,
                  fontWeight: FontWeight.bold) // TextStyle
              ), // Text
              Text(profile.courseAndSection ?? "Not Provided",style: TextStyle(color: Colors.
              white)),
              Text(profile.age?.toString() ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.hobby ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.studentId ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.email ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.favoriteSubject ?? "Not Provided",style: TextStyle(color: Colors.white)),
              InteractiveFavButton(),
              IconButton(
                icon: const Icon(Icons.delete, color: Colors.white),
                onPressed:  ()
                {
                  setState(() {
                    profiles.remove(profile);
                  });
                }
              ), // IconButton
              IconButton(
                icon: const Icon(Icons.hide_image, color: Colors.grey),
                onPressed: () {
                  setState(() {
                    profile.isNotHidden = !profile.isNotHidden;
                  });
                },
              ), // IconButton
              IconButton(
                icon: const Icon(
                  Icons.edit,
                  color: Colors.white,
                ), // Icon
                onPressed: () {
                  TextEditingController nameController =TextEditingController(text: profile.name);
                  TextEditingController hobbyController = TextEditingController(text: profile.hobby);
                  TextEditingController favsubController = TextEditingController(text: profile.
                  favoriteSubject);
                  showDialog(
                    context: context,
                    builder: (context) {
                      return AlertDialog(
                        title: const Text('Edit Student'),
                        content: Column(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            TextField(
                              controller: nameController,
```



```
                TextField(
                  controller: nameController,
                  decoration: const InputDecoration(
                    labelText: 'Name',
                  ), // InputDecoration
                ), // TextField
                TextField(
                  controller: hobbyController,
                  decoration: const InputDecoration(
                    labelText: 'Hobby',
                  ), // InputDecoration
                ), // TextField
                TextField(
                  controller: favsubController,
                  decoration: const InputDecoration(
                    labelText: 'Favorite Subject',
                  ), // InputDecoration
                ) // TextField
              ]
            ), // Column
            actions: [
              TextButton(
                onPressed: () {
                  Navigator.pop(context);
                },
                child: const Text('Cancel'),
              ), // TextButton
              TextButton(
                onPressed: () {
                  setState(() {
                    profile.name = nameController.text;
                    profile.hobby = hobbyController.text;
                    profile.favoriteSubject = favsubController.text;
                  });
                  Navigator.pop(context);
                },
                child: const Text('Save'),
              ), // TextButton
            ],
          ); // AlertDialog
        },
      );
    },
  ), // IconButton

]
), // Column
), // Card
) : // GestureDetector
IconButton(
  icon: const Icon(Icons.no_encryption_gmailerrorred_outlined, color: Colors.grey),
  onPressed: () {
    setState(() {
      profile.isNotHidden = !profile.isNotHidden;
    });
  },
) // IconButton
); // SizedBox
}
```

🏴 Flag 3 — The Empty Directory

Objective: Handle situations where there are no students to display.

Modify your Student Directory so it detects when the student list is empty.

Create an **empty state**.

Examples:

- "No students found"
- Empty-state icon
- Button to add a student

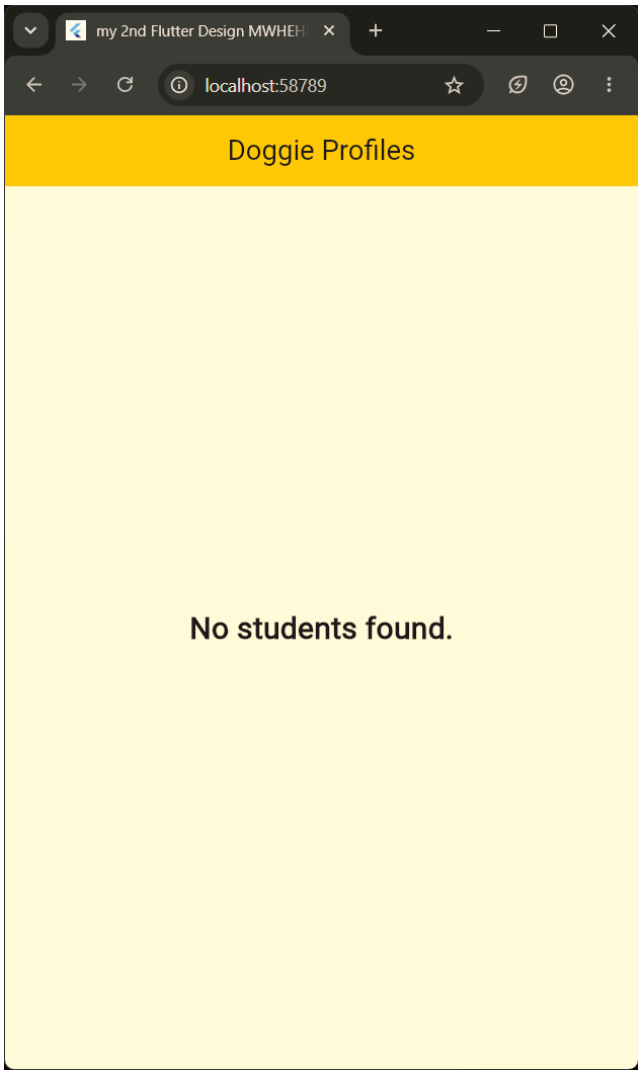
Test

Delete all students in your application (not the code)

Instead of showing a blank screen, your application should display your empty state.

Screenshot:

- Empty student list
- Empty-state UI
- Condition checking for an empty list



```dart
@override
Widget build(BuildContext context) {
  profiles.sort((a,b) => (a.name ?? "").compareTo(b.name ?? ""));
  return Scaffold(
      backgroundColor: const Color(0xFFFFF8E7),
      appBar: AppBar(
        title: const Text("Doggie Profiles"),
        backgroundColor: Colors.amber,
        centerTitle: true,
      ), // AppBar
      body: profiles.isEmpty
          ? const Center(
        child: Text(
          "No students found.",
          style: TextStyle(
            fontSize: 24,
            fontWeight: FontWeight.bold,
          ), // TextStyle
        ), // Text
      ) // Center
          : ListView.builder(
        itemCount: profiles.length,
        itemBuilder: (context, index) {
          return profilesOfEachUser(profiles[index]);
        },
      ), // ListView.builder
  ); // Scaffold
  }
}
```

🏴 Flag 4 — Loading the Directory

Objective: Display different UI while student data is being loaded.

Create a loading state for your Student Directory.

Research how Flutter can conditionally display a loading indicator.

Examples:

- `CircularProgressIndicator`
- Loading message
- Loading placeholder

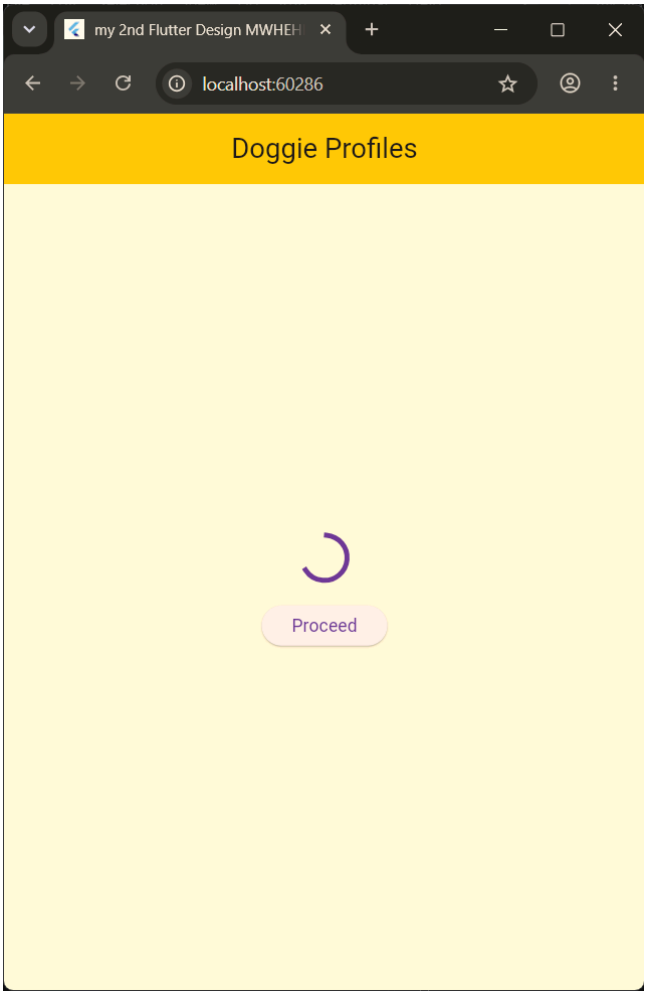
Test

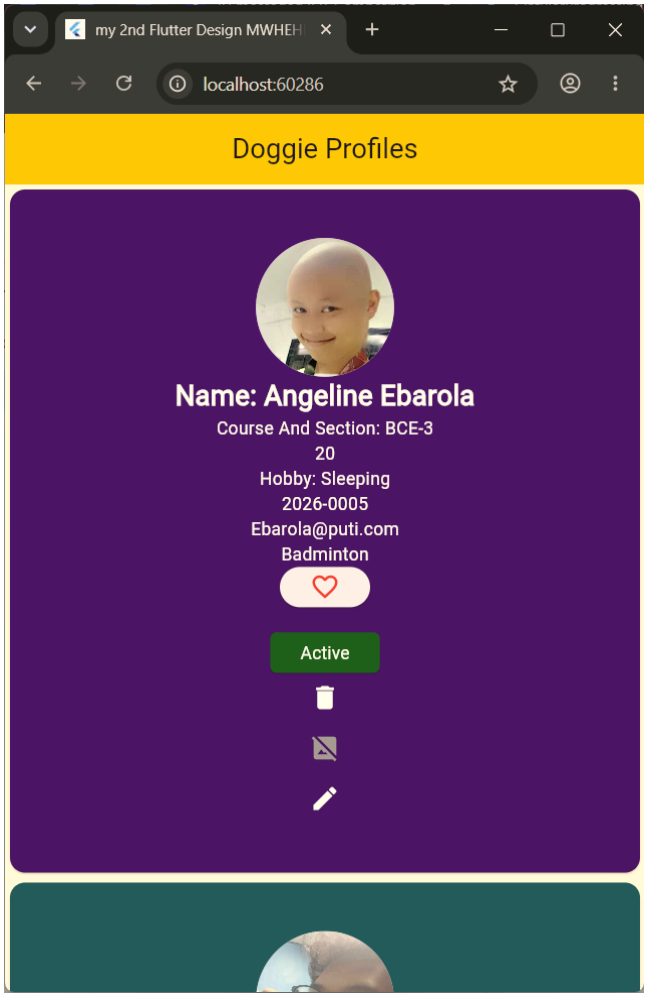
When the application is in a loading state, the student list should not be displayed as the main content.

Once loading is complete, the Student Directory should appear.

Screenshot:

- Loading state
- Loaded state
- Conditional loading logic





// I Added the proceed button maam since the data is in the memory directly, so retrieval is blink of an eye, so I added a button to pause it a bit

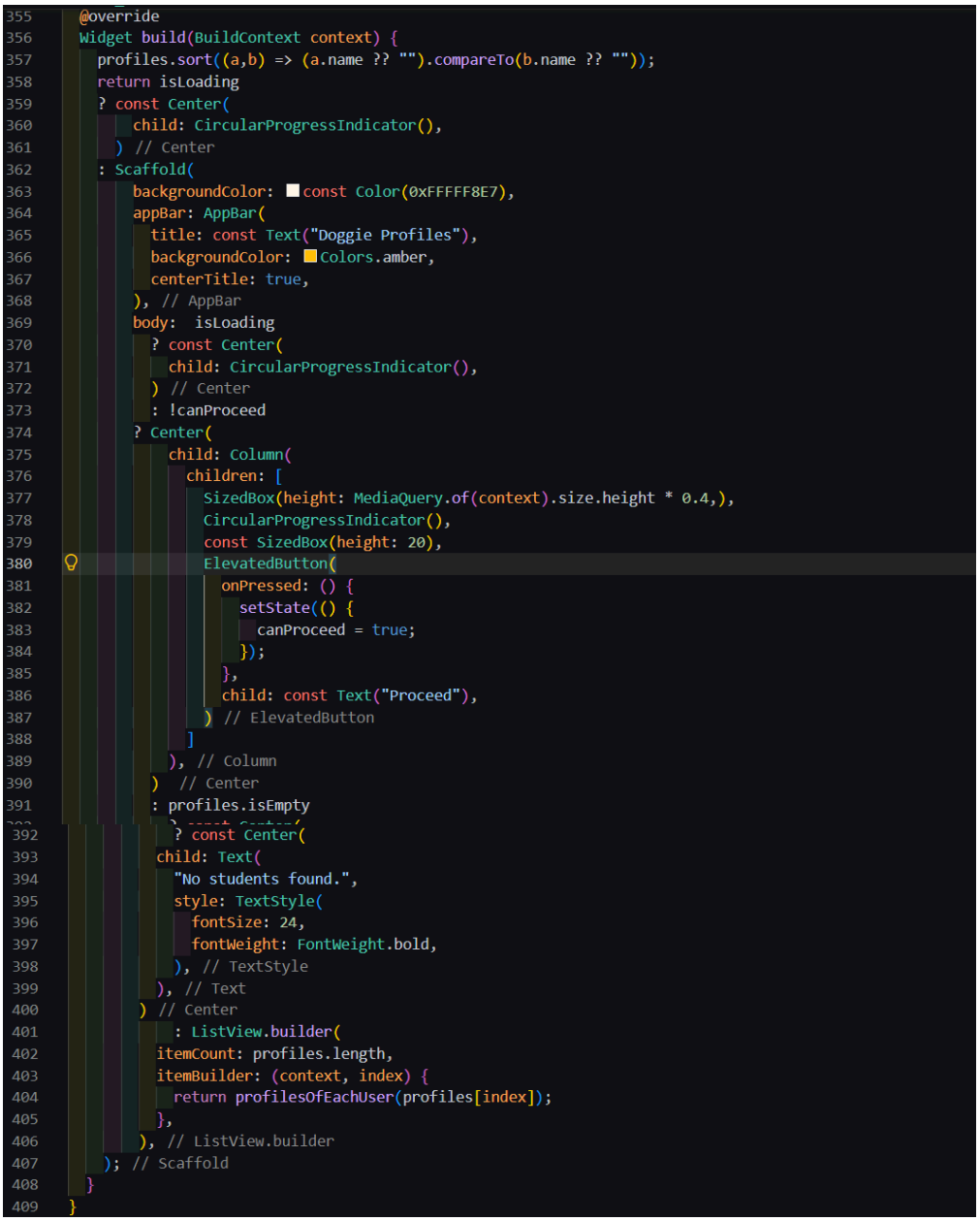
```dart
@override
void initState() {
  super.initState();
  loadProfiles();
}

Future<void> loadProfiles() async {
  setState(() {
    isLoading = true;
  });

  try {
    if (!mounted) return;

    setState(() {
      loadedProfiles = profiles;
      isLoading = false;
    });
  } catch (e) {
    if (!mounted) return;

    setState(() {
      isLoading = false;
    });
  }
}
```



```dart
@override
Widget build(BuildContext context) {
  profiles.sort((a,b) => (a.name ?? "").compareTo(b.name ?? ""));
  return isLoading
  ? const Center(
      child: CircularProgressIndicator(),
    ) // Center
  : Scaffold(
      backgroundColor: const Color(0xFFFFF8E7),
      appBar: AppBar(
        title: const Text("Doggie Profiles"),
        backgroundColor: Colors.amber,
        centerTitle: true,
      ), // AppBar
      body:  isLoading
        ? const Center(
          child: CircularProgressIndicator(),
        ) // Center
        : !canProceed
      ? Center(
          child: Column(
            children: [
              SizedBox(height: MediaQuery.of(context).size.height * 0.4,),
              CircularProgressIndicator(),
              const SizedBox(height: 20),
              ElevatedButton(
                onPressed: () {
                  setState(() {
                    canProceed = true;
                  });
                },
                child: const Text("Proceed"),
              ) // ElevatedButton
            ]
          ), // Column
        )  // Center
        : profiles.isEmpty
          ? const Center(
          child: Text(
            "No students found.",
            style: TextStyle(
              fontSize: 24,
              fontWeight: FontWeight.bold,
            ), // TextStyle
          ), // Text
        ) // Center
          : ListView.builder(
          itemCount: profiles.length,
          itemBuilder: (context, index) {
            return profilesOfEachUser(profiles[index]);
          },
        ), // ListView.builder
    ); // Scaffold
  }
}
```

🏴 Flag 5 — Multiple States

Objective: Make your Student Directory handle multiple UI states correctly.

Your application should now be able to display:

1. Loading
2. Students available
3. No students available

Determine which state should be displayed based on your data.

Test

Verify each state:

Loading
→ Loading UI

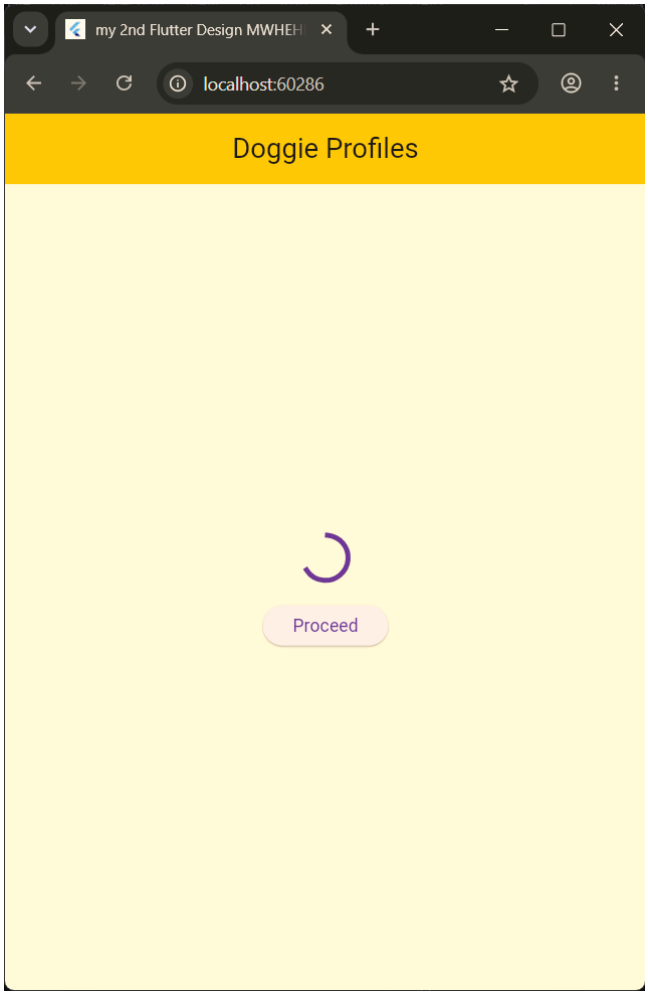
Students available
→ Student List

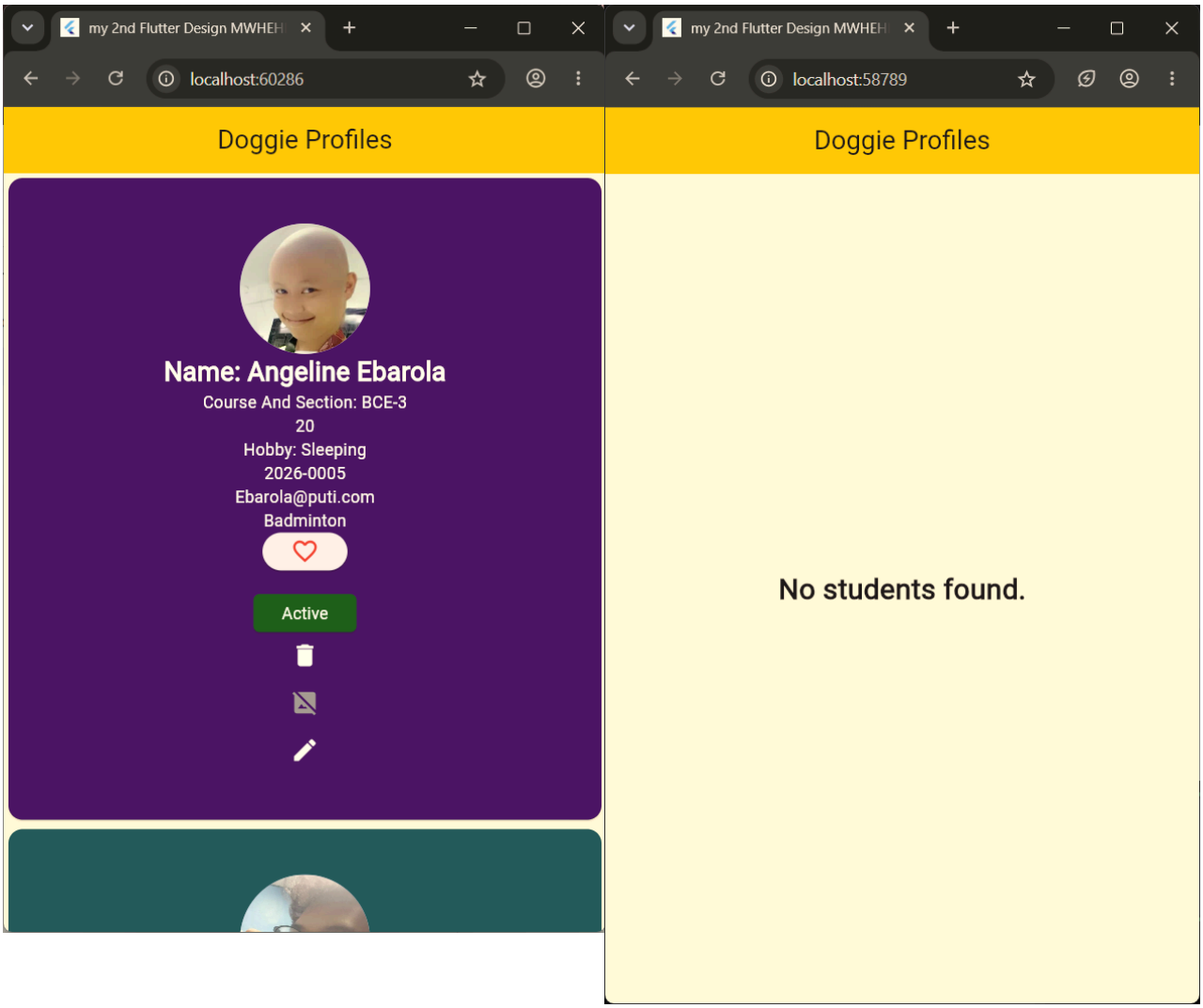
Empty list
→ Empty State

Only the appropriate UI should appear at a time.

Screenshot:

- Loading state
- Student list state
- Empty state




my 2nd Flutter Design MWHEH
localhost:60286
Doggie Profiles
Name: Angeline Ebarola
Course And Section: BCE-3
20
Hobby: Sleeping
2026-0005
Ebarola@puti.com
Badminton
Active
my 2nd Flutter Design MWHEH
localhost:58789
Doggie Profiles
No students found.

🏴 Flag 6 — States Work Together

Objective: Make sure conditional UI does not break your existing interactions.

Test the combination of:

- Favorite
- Delete
- Edit
- Active/Inactive
- Show/Hide
- Empty state
- Loading state

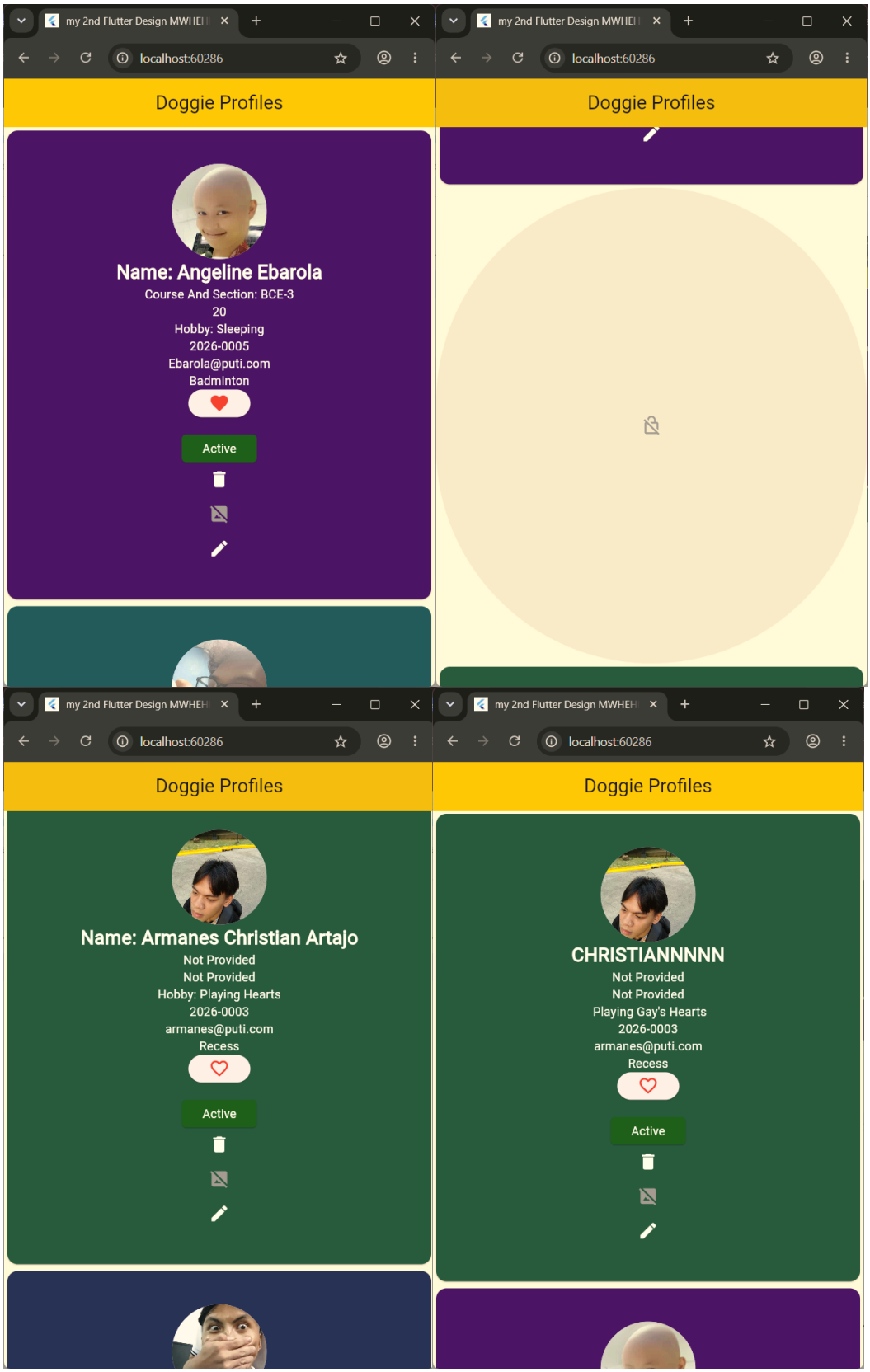
Test Scenario

1. Favorite a student.
2. Change or display their Active/Inactive state.
3. Delete students until the directory becomes empty.
4. Verify the Empty State appears.
5. Reload or restore students.
6. Verify the Student Directory appears again.

The application should continue functioning correctly as its state changes.

Screenshot:

- Multiple conditional states working
- Interactive Student Card
- Empty State
- Application functioning normally


my 2nd Flutter Design MWHEH
localhost:60286
Doggie Profiles
Name: Angeline Ebarola
Course And Section: BCE-3
20
Hobby: Sleeping
2026-0005
Ebarola@puti.com
Badminton
Active
Doggie Profiles
Name: Armanes Christian Artajo
Not Provided
Not Provided
Hobby: Playing Hearts
2026-0003
armanes@puti.com
Recess
Active
Doggie Profiles
CHRISTIANNNNN
Not Provided
Not Provided
Playing Gay's Hearts
2026-0003
armanes@puti.com
Recess
Active

my 2nd Flutter Design MWHEH

localhost:60286

Doggie Profiles



Name: Carl Joshua Santos

Not Provided
Not Provided
Hobby: Sleeping
2026-0001
carl@puti.com
Astronomy

Active

my 2nd Flutter Design MWHEH

localhost:60286

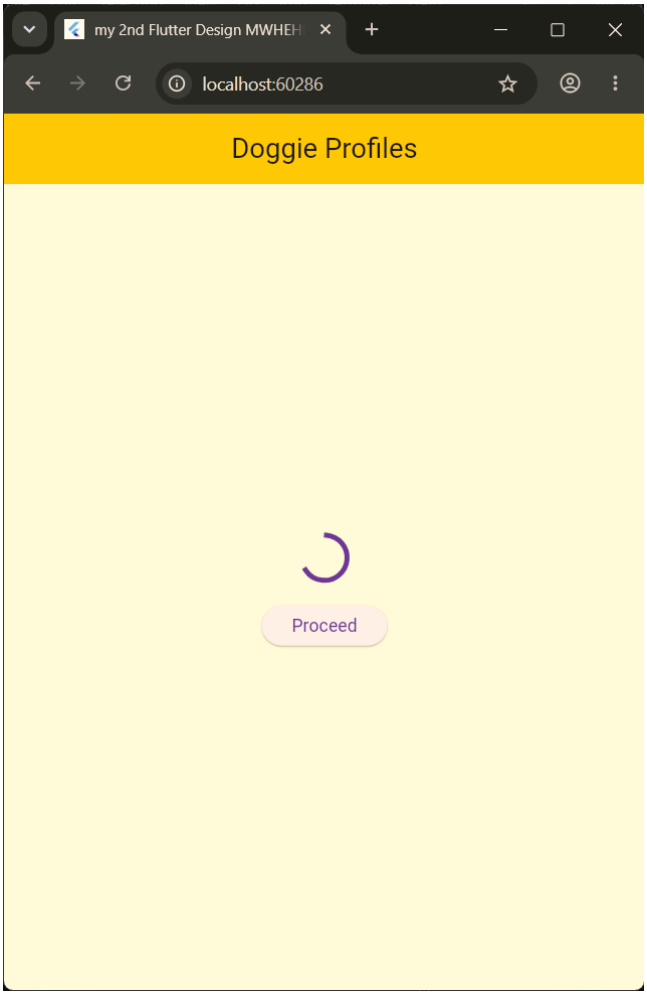
Doggie Profiles



Name: Armanes Christian Artajo

Not Provided
Not Provided
Hobby: Playing Hearts
2026-0003
armanes@puti.com
Recess

Active



🏴 Flag 7 — Save Your Progress

Objective: Commit and push your completed Conditional UI Student Directory.

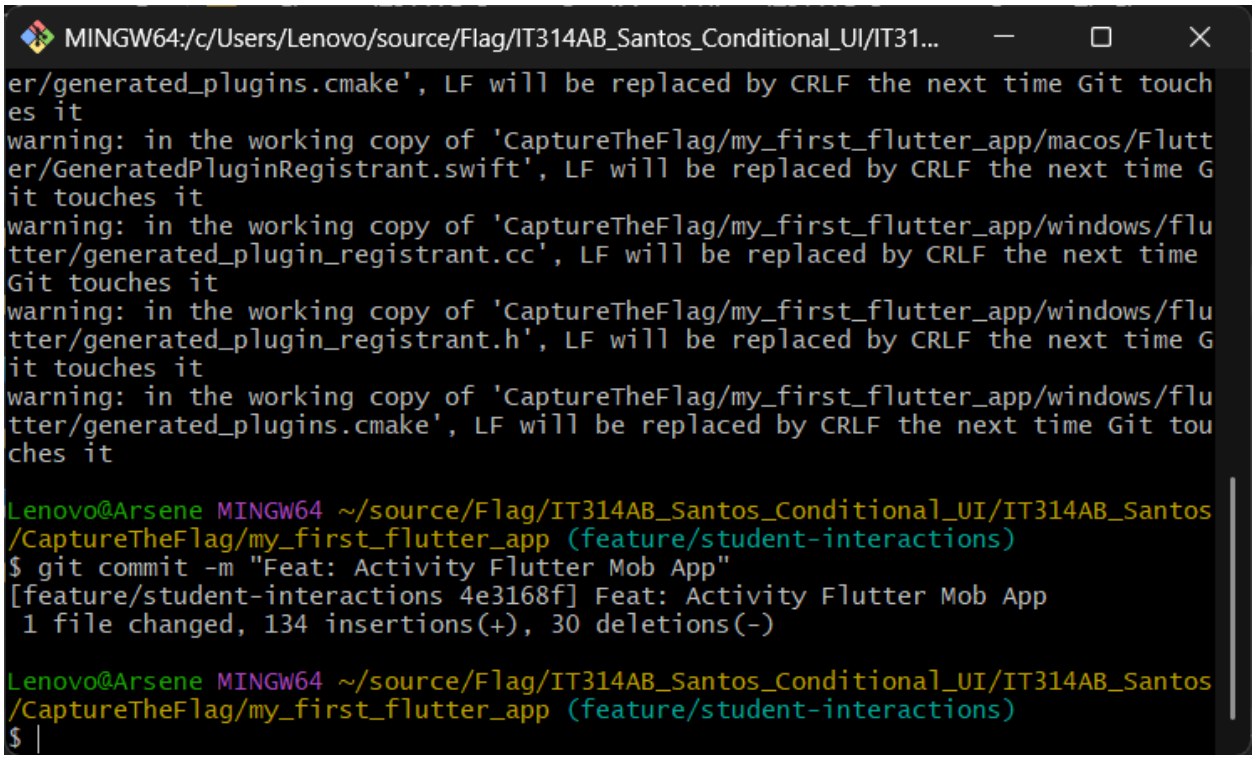
Use a proper Git commit message.

Branch:

`feature/conditional-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit


MINGW64:/c/Users/Lenovo/source/Flag/IT314AB_Santos_Conditional_UI/IT31...
er/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touch
es it
warning: in the working copy of 'CaptureTheFlag/my_first_flutter_app/macos/Flutt
er/GeneratedPluginRegistrant.swift', LF will be replaced by CRLF the next time G
it touches it
warning: in the working copy of 'CaptureTheFlag/my_first_flutter_app/windows/flu
tter/generated_plugin_registrant.cc', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'CaptureTheFlag/my_first_flutter_app/windows/flu
tter/generated_plugin_registrant.h', LF will be replaced by CRLF the next time G
it touches it
warning: in the working copy of 'CaptureTheFlag/my_first_flutter_app/windows/flu
tter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git tou
ches it
Lenovo@Arsene MINGW64 ~/source/Flag/IT314AB_Santos_Conditional_UI/IT314AB_Santos
/CaptureTheFlag/my_first_flutter_app (feature/student-interactions)
$ git commit -m "Feat: Activity Flutter Mob App"
[feature/student-interactions 4e3168f] Feat: Activity Flutter Mob App
1 file changed, 134 insertions(+), 30 deletions(-)
Lenovo@Arsene MINGW64 ~/source/Flag/IT314AB_Santos_Conditional_UI/IT314AB_Santos
/CaptureTheFlag/my_first_flutter_app (feature/student-interactions)
$